SDE INTERVIEW PRESENTATION GUIDE

AI RESEARCH ASSISTANT - ARCHITECTURE & CODE EXPLANATION

Topic:	Detailed Project Explanation	**Date:**	2026-05-30
Recipient:	Banao Technologies Recruiter / SDE Panel	**Status:**	Technical Deliverable (100% Verified)

1. The SDE Elevator Pitch (The 30-Second Hook)

"I designed and built an advanced, high-performance RAG-powered PDF Reading and Technical Document Understanding Assistant using a Python + Flask backend and a modern HTML/CSS/JS glassmorphic dark-theme UI. The project is powered by Groq's high-speed Llama-3.3-70B model. It solves a crucial academic and engineering problem: information overload and time inefficiency when scanning dense technical documents. Instead of manual copy-pasting, users get structured summaries, custom citation records, 3D flippable concept study cards, and grounded Q&A; within a highly optimized, cost-efficient, and secure local application."

2. System Architecture & Component Interactions

The application is engineered following clean Separation of Concerns (SoC) principles, structured into three highly decoupled modules:

- **Frontend Interface Layer (Templates)**: A single-page application (SPA) styled with HSL variables and backdrop blur filters. It renders markdown tables, lists, preformatted code blocks, and math notations using *marked.js*, maintaining scroll-locked grid structures.
- **Backend API Controller (Flask)**: Registers secure RESTful endpoints (/upload, /summarize, /query, /citation, /flashcards). It manages active user sessions and maps text caching on disk to bypass session cookie limits.
- **Model Reasoning Layer (Groq API)**: Connects dynamically to the *llama-3.3-70b-versatile* model, sending targeted prompt instructions to enforce grounded, hallucination-free generation.

3. Deep-Dive of Core SDE Implementations

A. Document Scrape & Validation Auditing

The scraping module uses PyMuPDF (fitz) to scrape text rapidly from PDFs. The system implements a scanned-document audit check: if the extracted word count is 0, the backend catches the anomaly, removes the empty file from disk to prevent storage bloat, and returns an error bubble recommendation. Extracted text is stored as a companion .txt file on the local filesystem, mapped by session keys, preventing session cookie capacity overflow.

B. Smart Chunking & Lexical Retrieval (RAG)

To overcome LLM context window boundaries and avoid expensive processing: if a PDF contains more than 6,000 words, it is split into overlapping sliding-window chunks (3,000 words each, 500-word overlap) to preserve continuity. When a user asks a question, a custom, pure-Python lexical overlap search ranker filters out stopwords, counts term frequencies in each chunk, and **retrieves only the single most relevant chunk** to send to Groq. This grounds responses, eliminates hallucination, and maintains latency under 1.5 seconds.

C. Interactive 3D Study Flashcards

Extracts exactly 6 crucial technical terms and simplified definitions from the active PDF in JSON format. On the frontend, these are rendered using 3D CSS animations. By setting 'perspective: 1000px' on the parent grid, 'transform-style: preserve-3d' on the card inner layer, and 'backface-visibility: hidden' on card faces, clicking any card triggers a hardware-accelerated 180-degree Y-axis rotation to reveal the definition.

D. Metadata Citation & BibTeX Generator

Extracts title, authors, journal, year, volume, and pages by scanning ONLY the first two pages of the PDF. The backend instructs Llama-3.3 to return a structured JSON, which Python algorithms format into a standardized APA Style string and a valid LaTeX BibTeX block, copyable with a single click.

4. Engineering Maturity & Robust Fail-Safes

A. Regular Expression JSON Extraction Filters

LLMs frequently prepend conversational filler text before and after JSON outputs (e.g. 'Here is your JSON...'), which crashes standard json.loads() calls. We solved this by implementing strict backend Regular Expression filters (re.search(r'\[.*\]', raw_content, re.DOTALL) for lists and re.search(r'\{.*\}', raw_content) for objects). This surgically isolates ONLY the structural JSON block, making the parser completely immune to conversational noise.

B. Case-Insensitive JavaScript Key Mappers

LLMs occasionally alternate key casing (e.g. 'Term' vs 'term'). The frontend JS includes key-normalization mapping const term = card.term || card.Term || card.concept... to dynamically resolve properties and prevent card-rendering crashes.

C. Viewport Grid Constraints (Scroll Lock)

Lacked height locks cause flexbox timelines to grow taller than the screen, pushing input boxes off-screen. We constrained container heights to exactly 100vh and hid parent grid overflow, forcing the timeline to scroll natively while keeping input elements securely anchored at the bottom.

5. SDE Interview Key Winning Points to Highlight

- **Data Privacy & Security**: Processing PDFs and hosting chat histories locally keeps proprietary documents secure on private servers, bypassing the privacy risks of public third-party web portals like ChatGPT.
- **Resource Efficiency**: Instead of installing heavy, expensive vector databases (like ChromaDB or Pinecone) and embedding APIs, we implemented a lightweight, pure-Python lexical ranker. This avoids architectural bloat and keeps the system completely free to run.
- **Structured Prompt Engineering**: Demonstrates high proficiency in guiding LLMs to yield consistent JSON schema structures, which we cleanly parse, map, and render programmatically.